Assignment 3: End-to-End Hugging Face Model Training & Docker Deployment

Mamta Chauhan
Roll No: M25CSA018
Subject: ML DL Ops

0.1 Objective

The objective of this assignment is to build a complete machine learning workflow starting from a Jupyter notebook, converting it into production-ready Python scripts, training and evaluating a model using Hugging Face tools, containerizing the workflow using Docker, and publishing artifacts to GitHub and Hugging Face.

This assignment demonstrates the integration of Machine Learning, DevOps, and production deployment practices (MLOps).

0.2 Project Links

GitHub Repository:
`https://github.com/mamta54/MLOps-Mamta-M25CSA018/tree/Assignment3`

Hugging Face Model:
`https://huggingface.co/mamta54/s3model`

0.3 Task 1: Notebook Setup

The provided notebook `ML_DL_Ops_Ass_3-Fine-Tuning-Classification.ipynb` was downloaded and placed inside the project working directory. The notebook served as the initial development prototype before being converted into modular production scripts.

0.4 Task 2: Docker Environment Setup

A Docker environment was created to ensure reproducibility, dependency isolation, and consistent execution across systems.

Base Image

The PyTorch CUDA runtime image was used for GPU compatibility:
`pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime`

Installed Dependencies

The following libraries were installed via `requirements.txt`:

- transformers
- datasets
- evaluate
- torch
- scikit-learn

- numpy
- pandas
- huggingface_hub

Docker ensured portability and reproducibility of the training and evaluation pipeline.

0.5 Task 3: Conversion to Production Scripts

The notebook was converted into modular Python scripts:

- `train.py`
- `evaluate.py`
- `data.py`
- `utils.py`

Notebook-specific artifacts (interactive cells, print debugging, and temporary outputs) were removed to make the code production-ready and structured for deployment.

0.6 Task 4: Model Selection

The pre-trained model selected was:
(TinyBERT)

Reason for Selection

- Lightweight architecture compared to BERT
- Faster training and inference
- Lower GPU memory consumption
- Strong contextual representation for text classification
- Suitable balance between computational efficiency and accuracy

0.7 Task 5: Training Using Trainer API

The Hugging Face **Trainer API** was used for structured and scalable training.

Trainer API Capabilities

- Automated training loop management
- Evaluation scheduling
- Metric computation
- Model checkpointing and saving
- Easy integration with Hugging Face Hub

Training Configuration

- Epochs: 3
- Batch Size: 16
- Evaluation Strategy: Per Epoch
- Optimizer: AdamW

0.8 Task 6: Evaluation Results

Two evaluation runs were performed: one after local training and one after loading the model from Hugging Face Hub.

Run 1 Results

- Evaluation Loss: 3.3054
- Accuracy: 13.06%
- Weighted F1 Score: 0.1283

Run 2 Results (After Reload from Hugging Face)

- Evaluation Loss: 3.2908
- Accuracy: 14.37%
- Weighted F1 Score: 0.1382

Observations

- Slight improvement in accuracy from 13.06% to 14.37%.
- The class `poetry` showed significantly higher precision and recall in the second run.
- Some classes exhibited low precision and recall, indicating possible class imbalance.
- The relatively low overall accuracy suggests the need for:
  - More epochs
  - Hyperparameter tuning
  - Data balancing techniques
  - Larger training dataset

0.9 Task 7: Model Deployment to Hugging Face

The trained model and tokenizer were pushed to the Hugging Face Hub using:

```
model.push_to_hub("mamta54/s3model")
tokenizer.push_to_hub("mamta54/s3model")
```

This ensures:

- Public accessibility
- Version control
- Easy reproducibility
- Centralized model hosting

0.10 Task 8: Re-evaluation from Hugging Face Repository

The model was reloaded directly from Hugging Face:

```
AutoModelForSequenceClassification.from_pretrained(
"mamta54/s3model")
```

The evaluation metrics closely matched the locally saved model, confirming reproducibility and correct deployment.

0.11 Task 9: Production Docker Image (Evaluation Only)

A final production-ready Docker image was created for evaluation-only deployment.
This container:

- Pulls the trained model from Hugging Face Hub
- Automatically executes evaluation on startup
- Simulates real-world inference deployment
- Ensures environment consistency

0.12 Task 10: GitHub Repository

The project repository includes:

- Source code (train.py, evaluate.py, data.py, utils.py)
- Dockerfile and evaluation Dockerfile
- requirements.txt
- README.md
- Evaluation results

GitHub provides version control, collaboration capability, and structured documentation.

0.13 Challenges Faced

- Docker image pulling issues due to institutional network restrictions.
- GPU configuration and compatibility setup.
- Authentication setup for Hugging Face inside Docker containers.
- Managing dependency consistency across environments.